Data Structure, Midterm

Student ID: *Double click here to fill the Student ID*

Name: *Double click here to fill the name*

A. Single Choice Questions (5% each, 60%)

(1) A singly linked list has only a `head` pointer (no `tail` pointer). What is the time complexity of appending a new node to the **END** of the list?

(A) $O(1)$ — only the head pointer needs updating

(B) $O(n)$ — must traverse the entire list to find the last node

(C) $O(\log n)$ — binary search can locate the last position

(D) $O(1)$ if the list is non-empty, $O(n)$ if the list is empty

(2-1) What is the output of the following Python code?

```python
class Fraction:
    def __init__(self, num, den):
        self.num = num
        self.den = den
    def __add__(self, other):
        new_num = self.num * other.den + other.num * self.den
        new_den = self.den * other.den
        return Fraction(new_num, new_den)
    def __repr__(self):
        return f"{self.num}/{self.den}"

f1 = Fraction(1, 2)
f2 = Fraction(1, 3)
print(f1 + f2)
```

(A) `1/2`

(B) `5/6`

(C) `2/5`

(D) Error — `+` is not defined for custom classes

(2-2) What is the output of the following C++ code?

```cpp
#include <iostream>
using namespace std;

class Fraction {
public:
    int num, den;
    Fraction(int n, int d) : num(n), den(d) {}
    Fraction operator+(const Fraction& other) const {
        return Fraction(num * other.den + other.num * den, den * other.den);
    }
    friend ostream& operator<<(ostream& os, const Fraction& f) {
        return os << f.num << "/" << f.den;
    }
};

int main() {
    Fraction f1(1, 2), f2(1, 3);
    cout << (f1 + f2) << "\n";
}
```

(A) `1/2`

(B) `5/6`

(C) `2/5`

(D) Error — `operator+` is not defined for custom classes

(3) Which of the following statements about **Abstract Data Types (ADT)** is **TRUE**?

(A) An ADT specifies both the operations it supports **and** exactly how those operations must be implemented internally.

(B) A Stack ADT implemented with a Python `list` and one implemented with a linked list are **different** ADTs because they use different internal structures.

(C) An ADT defines **what** operations are available and **what** they do, completely independent of any implementation; a Stack ADT is valid as long as it obeys LIFO behavior regardless of the underlying storage.

(D) An ADT and a concrete data structure are the same concept — the term "ADT" is simply an older name for what we now call a "data structure".

(4-1) What is the average-case time complexity of the `in` operator for a Python `list` vs a Python `dict`?

(A) $O(n)$ for list, $O(1)$ for dict

(B) $O(1)$ for list, $O(n)$ for dict

(C) $O(1)$ for both

(D) $O(n)$ for both

(4-2) What is the average-case time complexity of `std::find()` on `std::vector` vs `count()` on `std::unordered_map`?

(A) $O(n)$ for vector, $O(1)$ for unordered_map

(B) $O(1)$ for vector, $O(n)$ for unordered_map

(C) $O(1)$ for both

(D) $O(n)$ for both

(5) Which of the following is NOT a valid postfix expression?

(A) `5 3 + 2 *`

(B) `A B + C D - *`

(C) `* A B + C`

(D) `A B C + * D -`

(6) When converting decimal 53 to binary using repeated division by 2, remainders are pushed onto a Stack as they are produced. After ALL divisions are complete (before any pops), what are the stack contents from bottom to top?

(A) `1, 1, 0, 1, 0, 1`

(B) `0, 1, 0, 1, 0, 0`

(C) `1, 0, 1, 0, 1, 1`

(D) `1, 1, 0, 0, 1, 1`

(7) In a doubly linked list, given only a direct reference to the node to be deleted (no head pointer needed), what is the time complexity of the deletion operation?

(A) $O(1)$

(B) $O(n)$ because you need to find the previous node

(C) $O(\log n)$

(D) $O(n^2)$

(8) What is the result of evaluating the prefix expression `* + 2 3 - 7 4`?

(A) 5

(B) 15

(C) 17

(D) 25

(9-1) What is the output of the following Python code?

```python
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1

a = Counter()
b = Counter()
c = Counter()
print(Counter.count)
print(a.count)
```

(A) `3` / `3`

(B) `1` / `1`

(C) `3` / `1`

(D) `0` / `0`

(9-2) What is the output of the following C++ code?

```cpp
#include <iostream>
using namespace std;

class Counter {
public:
    static int count;
    Counter() { count++; }
};
int Counter::count = 0;

int main() {
    Counter a, b, c;
    cout << Counter::count << "\n";
    cout << a.count << "\n";
}
```

(A) `3` / `3`

(B) `1` / `1`

(C) `3` / `1`

(D) `0` / `0`

(10) In the palindrome checker using a Deque, all characters are added to the rear one at a time. How are they then used to check if the string is a palindrome?

(A) Remove from front and rear simultaneously; it is a palindrome if all pairs match and the deque is empty (or has one element for odd-length strings)

(B) Remove all from front and compare with the original string reversed

(C) Remove from rear only and compare character by character

(D) Push all to a Stack then compare with the Deque

(11) A Queue is initialized empty. The following operations are performed in order: `enqueue(5)`, `enqueue(3)`, `dequeue()`, `enqueue(7)`, `enqueue(1)`, `dequeue()`. What are the contents of the Queue from **front to rear** after all operations complete?

(A) `[7, 1]`

(B) `[5, 7, 1]`

(C) `[3, 7]`

(D) `[1]`

(12) Which of the following statements about **Big O notation** is **FALSE**?

(A) Big O notation ignores constant factors and lower-order terms: $5n^2 + 100n + 3$ and n^2 are both $O(n^2)$.

(B) Two algorithms with the same Big O time complexity — for example, both $O(n^2)$ — will always execute the **same number of steps** and therefore always take the **same amount of time** for any input size n.

(C) An algorithm that performs n^2 steps for every input of size n has time complexity $O(n^2)$.

(D) Big O notation describes how an algorithm's resource usage **scales** as the input size grows, not the exact number of steps.

B. Short-answer questions, Please provide the derivation for each question along with your answer (8% each, 40%).

(13) Briefly explain each of the following terms.

(a) Method Overloading

(b) Row-Major Order

(c) Deque

(d) Infix Expression

(14-1) Evaluate the time complexity of the following code segment in terms of Big-O notation:

```
i = 1
while i <= n:
    for j in range(i):
        x += 1
    i *= 3

for i in range(n):
    for j in range(1, n + 1, 2):
        x -= 1
```

(14-2) Evaluate the time complexity of the following code segment in terms of Big-O notation:

```
int i = 1;
while (i <= n) {
    for (int j = 0; j < i; j++) x++;
    i *= 3;
}

for (int i = 0; i < n; i++) {
    for (int j = 1; j <= n; j += 2) x--;
}
```

(15) Convert the infix expression `A * B - (C + D) / E` to postfix using a stack, as described in class.

(a) Show the state of the operator stack and output queue after processing EACH token.

(b) What is the final postfix expression?

(c) What is the maximum number of symbols in the stack at any point during the conversion?

(16) Fill in the time complexity table below. Then briefly explain ONE scenario where an Unordered Linked List would be preferred over a Dynamic Array (Python list).

Operation	Unordered Linked List	Dynamic Array (Python list)
Insert at beginning	?	?
Delete a specific element (value given)	?	?
Access by index	?	?
Search for an element	?	?

(17) A student grade array stores data in column-major order, where rows represent students and columns represent assignments.

- `Grade[2][3]` is located at address **252**
- `Grade[5][1]` is located at address **226**
- `Grade[0][2]` is located at address **232**

(a) What is the base address of `Grade[0][0]`?

(b) What is the size of each element (in bytes)?

(c) How many students (rows) does this array have?

(d) What is the address of `Grade[7][5]`?

Bonus: Sparse Matrix (10%)

A **sparse matrix** stores only non-zero elements to save memory. Two common formats:

- **COO (Coordinate)**: Three parallel arrays each of length NNZ (number of non-zeros).
- **DOK (Dictionary of Keys)**: Map `(row, col)` tuples to values.

Given the following 4×4 matrix M:

```
0  0  3  0
0  0  0  0
7  0  0  2
0  5  0  0
```

(a) Represent M in COO format. Show the three arrays: `row_indices`, `col_indices`, `values`.

(b) Represent the same matrix using DOK format. What data structure is used?

(c) A dense 4×4 matrix uses 4 bytes per element. How many bytes does the dense format use? How many bytes does the COO format use (assuming each array element is also 4 bytes)?

(d) Under what condition (in terms of NNZ and matrix size $n \times m$) is the COO format more memory-efficient than the dense format?